Limits and Precision

Real numbers go on forever. Think of a number, the biggest you can imagine, then add one. See? (No, infinity does not count.) Computers, the electronic kind we know and love, do not possess arbitrarily large amounts of storage. There are limits on how big numbers can get before the limits of system architecture are hit.

There are (courtesy of the IEEE) standards concerning the representation of numbers in computer memory. Different combinations of hardware and operating systems mean there are differences in system limitations. PHP provides tools for accessing information about those limits.

Integers

Depending on the platform and PHP version, a PHP integer will either be 32 or 64 bits long (or alternatively 4 or 8 bytes long) and always signed (which means one of those bits is used to indicate if an int is negative or not). This means the maximum range of a PHP integer is either *-2,147,483,647* to *2,147,483,647* or *-9,223,372,036,854,775,808* to *9,223,372,036,854,775,808*. We can find out at run time what the integer limits are by using some built-in constants:

```
print PHP_INT_SIZE . PHP_EOL;
print PHP_INT_MAX . PHP_EOL;
print PHP_INT_MIN . PHP_EOL;
```

```
Output:
8
9223372036854775807
-9223372036854775808
```

Most computers today have 64-bit processors, and if your PHP binary is also 64-bit, then the value of `PHP_INT_MAX` will be the maximum signed integer value, and `PHP_INT_SIZE` will be "8" (because an IEEE 64-bit integer uses 8 bytes of memory). A 32-bit PHP binary running on a 64-bit system will have `PHP_INT_SIZE` of "4" regardless of hardware.